

FirstKnock Pipeline — Consolidated Implementation Document

FirstKnock Pipeline — Consolidated Implementation Document

Production Developer Reference — Zero-Cost Classification Pipeline

April 2026

Document Type	Production Developer Reference
Status	Ready for Implementation
Supersedes	All prior pipeline architecture documents
Date	April 7, 2026

This document is the single authoritative reference for the FirstKnock data pipeline. It consolidates the Revised Pipeline Architecture Spec and the Full Pipeline Audit Report into one implementation-ready document. All seven sections must be read in order before deployment. Do not reference superseded documents.

CRITICAL: Four P0/P1 bugs are confirmed in the current pipeline. Two are silent failures — they produce no errors but generate incorrect classifications at runtime. All P0 fixes must be applied before any deployment proceeds.

~215Records/Area Sent to BatchData (Current)	~21Records/Area After Fix (Target)	90%BatchData Cost Reduction	\$172rightarrow\$17 Monthly Cost/Rep (8 Areas)
--	------------------------------------	-----------------------------	--

Section 1 — County Deed Lag Research Findings

Research Methodology

County deed recording lag has two distinct components that must both be accounted for when configuring the pipeline's lag window cutoff:

1. County Clerk Recording Lag — the number of days from property closing to the deed being recorded at the county clerk's office. This varies by county volume, submission method

(e-recording vs. mail/in-person), and seasonal backlog.

- 2. **RentCast Data Propagation Lag** — the number of days from deed recorded at the county to that deed appearing in the RentCast /properties API. RentCast sources data through major aggregators (likely ATTOM Data Solutions and/or CoreLogic) rather than direct county feeds, adding a second layer of delay on top of clerk lag.

The combined worst-case lag across all five target counties drives the recommended DEED_LAG_CUTOFF_DAYS value. Underestimating this lag causes recently-sold properties to be misclassified as EXPIRED_WITHDRAWN, generating false positives and wasted BatchData calls.

Per-County Lag Table

Table 1 — Per-County Deed Recording and RentCast Propagation Lag Estimates

County	Clerk Recording Lag	RentCast Propagation Lag	Total Estimated Lag	Notes
Doña Ana County, NM(Las Cruces — ZIPs 88001, 88005, 88012)	1–10 business days(1–2 e-recording;7–10 mail/in-person)	30–60 days	35–75 days	Lower-volume county; e-recording common; RentCast likely via ATTOM/CoreLogic aggregator with 48-hr raw refresh but batch propagation adds weeks
Bernalillo County, NM(Albuquerque)	3–10 business days	30–60 days	35–75 days	Mid-volume county; no official backlog disclosures; similar NM processing profile
Harris County, TX (Houston)	1–10 business days(1–2 e-recording;up to 60–90 days during peak backlog)	45–90 days	50–120 days	Highest-volume county in dataset; known backlog periods stretch to 2–3 months; worst-case scenario drives 120-day recommendation

Travis County, TX (Austin)	3–10 business days	30–60 days	35–75 days	Mid-volume; Austin market active; no official backlog
Bexar County, TX (San Antonio)	3–10 business days	30–60 days	35–75 days	Mid-volume; no official backlog disclosures

Key Research Findings

County Clerk Recording Lag

- Texas and New Mexico have no statutory deadline for recording a deed after closing; timely recording is industry practice but not legally mandated within a fixed window.
- E-recording (electronic submission) reduces clerk lag to 1–2 business days in most counties.
- Mail/in-person submissions: 7–10 business days typical; up to 2–3 months during peak backlog periods in high-volume counties.
- Harris County, TX is the highest-volume county in this dataset and the confirmed worst-case scenario. During peak real estate seasons, Harris County recording backlogs have stretched to 60–90 days.
- No official processing time disclosures or press releases have been published by any of the five target county clerks — lag estimates are based on industry benchmarks and observed pipeline behavior.

RentCast Data Propagation Lag

- RentCast does not publicly disclose its data refresh cadence or deed propagation timelines.
- RentCast sources property data through major aggregators (likely ATTOM Data Solutions and/or CoreLogic) rather than direct county feeds.
- CoreLogic refreshes property attributes every 24 hours; ATTOM refreshes every 48 hours — but these are raw database refresh rates, not end-to-end propagation times.
- End-to-end propagation (county recording → aggregator ingestion → RentCast API) adds 30–90 days in practice based on observed pipeline behavior.
- Industry research indicates that data latency exceeding 24 hours can reduce predictive performance of real estate AI and valuation models by 12–18%, underscoring the importance of accounting for this lag in classification logic.

Table 2 — Industry Aggregator Refresh Benchmarks (Source: Industry Research)

Data Aggregator	Refresh Cycle	US Property Coverage	Data Attributes	Relevance to Pipeline
CoreLogic	24 hours	98%	200+	Likely primary aggregator for RentCast; fastest raw refresh but end-to-end propagation still adds weeks
ATTOM Data	48 hours	95%	150+	Secondary aggregator candidate; 48-hr general refresh; daily for foreclosure data; monthly for permit data

Recommended DEED_LAG_CUTOFF_DAYS

Recommendation: 120 days. This is the single most important configuration value in the pipeline. Setting it too low causes false EXPIRED_WITHDRAWN classifications for recently-sold properties in high-lag counties.

Justification for the 120-day recommendation:

- Harris County worst-case clerk recording lag: up to 60–90 days during peak backlog
- RentCast propagation lag on top of clerk lag: 30–60 additional days
- Combined worst-case: 90–120+ days
- A 90-day cutoff would incorrectly classify recently-sold Harris County properties as EXPIRED_WITHDRAWN during backlog periods, generating false positives and unnecessary BatchData calls
- 120 days provides a safety buffer that covers the 95th-percentile lag scenario across all five target counties
- The cost of a false EXPIRED_WITHDRAWN classification (wasted BatchData call + missed sold signal) exceeds the cost of waiting an extra 30 days

Global Env Var vs. Per-County Config

Recommendation: Single global environment variable (DEED_LAG_CUTOFF_DAYS) with a 120-day default.

- Per-county config adds operational complexity — 5 env vars, county-code mapping logic, expanded deployment surface area — with marginal benefit given the conservative 120-day

default.

- The 120-day default safely covers all five counties including Harris County worst-case.
- If a specific deployment targets only lower-volume NM counties (Doña Ana, Bernalillo), the operator can override DEED_LAG_CUTOFF_DAYS=90 at deployment time.
- The env var approach preserves per-deployment flexibility without requiring code changes.

Section 2 — Environment Variables Required

All environment variables must be set before deployment. Variables marked Required will cause startup failure or silent data errors if missing. Set all variables in your Base44 environment configuration before running any migration or deployment step.

Table 3 — Required and Optional Environment Variables

Variable	Required	Default	Description
RENTCAST_API_KEY	Required	—	RentCast API authentication key. Used in processFetchChunk.ts for all /properties endpoint calls. Obtain from RentCast dashboard.
BATCHDATA_API_KEY	Required	—	BatchData API authentication key. Used for async bulk property lookup on AMBIGUOUS records. Obtain from BatchData
DEED_LAG_CUTOFF_DAYS	Required	120	Days since listing removal after which a property is classified EXPIRED_WITHDRAWN without BatchData verification. Default 120 covers Harris County worst-case backlog + RentCast propagation lag. NM-only deployments may use 90.

PIPELINE_SECRET	Required	—	Shared secret for BatchData webhook authentication. Must match the secret configured in BatchData webhook settings. Used in batchDataWebhookCallback.ts to verify incoming webhook
BATCHDATA_WEBHOOK_URL	Required	—	Public Base44 endpoint URL for the batchDataWebhookCallback function. Provided to BatchData when submitting async bulk jobs so BatchData knows where to POST results.
DATABASE_URL	Required	—	PostgreSQL connection string. Format: postgresql://user:password@host:port/dbname. Used by all pipeline functions for DB
DEFAULT_MARKET_TYPE	Optional	default	Controls sub-circle radius and grid layout for geographic coverage. Values: dense_suburban (tighter grid, smaller radius), sparse_rural (wider grid, larger radius), default (standard settings).

Never commit API keys or secrets to source control. All Required variables must be injected via your deployment environment (Base44 env config, AWS Secrets Manager, or equivalent). A missing RENTCAST_API_KEY or DATABASE_URL will cause silent data errors, not startup crashes — verify all variables are present before each deployment.

Section 3 — Deployment Order

Follow this exact sequence. Do not deploy out of order — later steps depend on earlier ones. Deploying processFetchChunk.ts before migrations are complete will cause runtime DB errors.

Step 1 — Run SQL Migrations (in order)

Execute migrations in this exact sequence against your PostgreSQL database. Each migration may reference tables created by the prior migration — order is mandatory.

Table 4 — SQL Migration Execution Order

Order	Migration File	Table Created	Purpose
001	001_listing_seen_log.sql	listing_seen_log	Tracks which listingIds have been processed; prevents reprocessing in delta validation (Step 2c)
002	002_heuristic_score_log.sql	heuristic_score_log	Stores per-listing heuristic scores for audit trail and shadow mode recalibration
003	003_pending_confirmation_queue.sql	pending_confirmation_queue	Holds AMBIGUOUS records awaiting BatchData webhook response or Redfin cross-check
004	004_job_runs.sql	job_runs	Tracks pipeline run metadata, chunk counts, cost metrics, and monitoring counters

Step 2 — Deploy scoreInactiveListing.ts

- Dependencies: None (no external API calls; DATABASE_URL required for logging only)
- Purpose: Heuristic scoring engine that classifies inactive listings into score buckets
- Verify: Unit tests pass; scoring logic returns expected score ranges for test fixtures; auto-reject logic fires correctly for DOM values near 90, 180, and 365 days

Step 3 — Deploy batchDataWebhookCallback.ts

- Dependencies: PIPELINE_SECRET and BATCHDATA_WEBHOOK_URL must be set
- Purpose: Receives async webhook callbacks from BatchData with bulk lookup results
- Verify: Webhook endpoint returns 200 on valid payload with correct PIPELINE_SECRET; returns 401 on invalid secret; upserts to property_validation_cache correctly

Step 4 — Deploy processFetchChunk.ts (rewrite)

- Dependencies: RENTCAST_API_KEY, DATABASE_URL, all four migrations complete
- Purpose: Core pipeline function — fetches /properties data, runs heuristic scoring, routes AMBIGUOUS records to BatchData
- Verify: Test run on single ZIP code; confirm saleDateRange=180 in all API calls; confirm zero references to statusCategory or soldPrice; confirm pagination terminates correctly

Step 5 — Shadow Mode (1–2 Weeks)

Run the heuristic pipeline but continue sending ALL inactive records to BatchData (not just AMBIGUOUS). Log heuristic scores alongside BatchData results in heuristic_score_log. Compare: where does the heuristic disagree with BatchData? Use the discrepancy log to recalibrate scoring weights before cutover.

- Duration: 1–2 weeks minimum
- Success criteria: False positive rate < 15%; heuristic agrees with BatchData on geq85% of records
- Do not proceed to Step 6 until success criteria are met

Step 6 — Partial Cutover

- Trust Phase 1 matches (Step 2a — hard signal within 45 days) → classify SOLD, skip BatchData
- Trust extreme scores (geq4 or leq-3) → classify directly, skip BatchData
- Send middle scores (-2 to 3, excluding Phase 1 matches) to BatchData
- Monitor: BatchData call volume should drop ~50–60% vs. shadow mode baseline

Step 7 — Full Automation

BatchData receives only AMBIGUOUS records (heuristic score -1 to 0). All other classifications are handled by the heuristic pipeline. Expected BatchData call volume: ~10% of prior baseline (~21 records/area vs. ~215).

Section 4 — P0 Fix Checklist

These are breaking bugs that cause silent data errors. The pipeline will appear to run without errors but will produce incorrect classifications for 100% of records until these fixes are applied. Complete all P0 fixes before any deployment.

Grep Commands — Run These First

Run the following grep commands against your source directory. All commands must return zero hits before deployment proceeds:

- `grep -r "statusCategory" ./src --include="*.ts" rightarrow MUST return zero hits`
- `grep -r "soldPrice" ./src --include="*.ts" rightarrow MUST return zero hits`
- `grep -r "saleDateRange=90" ./src --include="*.ts" rightarrow MUST return zero hits`
- `grep -r "saleDateRange: 90" ./src --include="*.ts" rightarrow MUST return zero hits`
- `grep -r "saleDateRange: '90'" ./src --include="*.ts" rightarrow MUST return zero hits`

Fix 1 — statusCategory (Breaking — Silent Failure)

Table 5 — Fix 1: statusCategory Field Removal

Attribute	Detail
Field	statusCategory
Exists in RentCast API?	NO — confirmed absent from all endpoints including /listings/sale and /properties
Failure Mode	Silent. No error is thrown. Code evaluates against undefined and produces incorrect classifications for every record.
Fix	Remove entirely. Replace all classification logic that depended on statusCategory with the heuristic scoring model in scoreInactiveListing.ts.
Valid RentCast status values	Active and Inactive only. No Sold, Pending, Expired, or Withdrawn values exist in the RentCast
Impact if not fixed	API. 100% of listings are misclassified. Pipeline appears healthy but outputs are garbage.

Fix 2 — soldPrice (Breaking — Silent Failure)

Table 6 — Fix 2: soldPrice Field Removal

Attribute	Detail
Field	soldPrice
Exists in RentCast API?	NO — confirmed absent from all listing endpoints. Only price (list price) exists on /listings/sale.
Failure Mode	Silent. Any code reading soldPrice from a listing record receives undefined. Sold price comparisons silently fail.
Fix	Remove entirely. Use lastSalePrice from /properties endpoint only, and only on property records (not listing records).

Priority	Issue	Impact	Effort	Status
P0	Remove all statusCategory references	Fixes silent classification failure — 100% of records affected	Low	Fix before deployment
P0	Remove all soldPrice references from listing logic	Fixes silent classification failure — corrupts delta validation	Low	Fix before deployment
P1	Implement Lag Window Rule (DEED_LAG_CUT OFF_DAYS	Reduces BatchData input ~40%	Medium	Implement in processFetchChunk.ts
P1	auto-reject) Implement Delta Validation (listingId deduplication)	Reduces BatchData input ~94% after first cycle	Medium	Requires listing_seen_log migration
P1	Implement heuristic scoring model	Reduces BatchData input to ~10% of pool	High	scoreInactiveListing.ts
P2	Migrate BatchData to async bulk	Reduces per-call overhead; enables webhook pattern	Medium	batchDataWebhookCallback.ts
P2	Add cross-chunk deduplication in processFetchChunk	Fixes duplicate records from overlapping sub-circles	Low	Deduplicate by formattedAddress before merge
P3	Add Redfin cross-check for Pending confirmation	Reduces false positives on under-contract properties	High	Optional enhancement; pending_confirmation_queue

Section 5 — Monitoring Alerts

Configure these alerts in your monitoring system (Datadog, CloudWatch, PagerDuty, or equivalent). All thresholds are per pipeline run. Alerts marked Critical require immediate pipeline halt and manual review.

Table 9 — Monitoring Alert Thresholds and Response Playbook

Alert Metric	Threshold	Severity	Meaning	Response
batchdata_calls_per_run	> 30	High	Upstream heuristic filter regression — model is not filtering correctly and routing too many records to BatchData	Review recent scoreInactiveListing.ts changes; check if scoring weights were modified; verify AMBIGUOUS bucket is not expanding
false_positive_rate	> 15%	High	Heuristic model degradation — model is misclassifying sold properties as EXPIRED_WITHDRAWN at unacceptable rate	Recalibrate scoring weights using heuristic_score_log; compare recent market conditions to training data
cost_per_area	> \$5.00	Medium	Cost regression — BatchData spend per area exceeds budget threshold	Check if saleDateRange was reduced back to 90; verify BatchData routing logic; confirm AMBIGUOUS bucket size
failed_chunks	> 0 after retry pass	High	Coverage gap — one or more geographic chunks failed to process even after retry	Manual review required for affected area; check RentCast API rate limits and error logs
duplicate_count	=== 0 on multi-chunk job	Critical	Dedup logic broken — all records may be passing through as duplicates, or dedup is not running	Halt pipeline; inspect merge phase; check listing_seen_log table for recent inserts

total_fetched === total_inserted	on multi-chunk job	Critical	Dedup not running — every fetched record is being inserted without dedup check	Halt pipeline; check merge phase logic; verify listing_seen_log lookups are executing
-------------------------------------	--------------------	----------	--	--

Critical-severity alerts require immediate pipeline halt. Do not allow the pipeline to continue processing when deduplication is confirmed broken — duplicate BatchData calls will inflate costs and corrupt classification records.

Section 6 — Expected Cost Impact

Cost Comparison Table

~\$172/moCurrent Cost Per Rep (8 Areas)	~\$17/moTarget Cost Per Rep (After Fix)	90%Cost Reduction	25+ repsGrowth Tier Break-Even
--	--	-------------------	-----------------------------------

Table 10 — BatchData Cost Comparison: Current vs. Optimized (at \$0.10/record)

Scenario	Records/Area	Cost/Area	Areas/Month	Monthly Cost/Rep
Current (broken — full Inactive pool to BatchData)	~215	~\$21.50	8	~\$172.00
After P0+P1 fixes (heuristic filter active)	~21	~\$2.10	8	~\$16.80
Reduction	~90%	~90%	—	~90%

Important Clarification: P0 vs. P1 Cost Impact

P0 fixes alone (removing statusCategory and soldPrice) do NOT reduce cost. P0 fixes correct silent failures — they make the pipeline produce correct outputs instead of silently wrong ones. Before P0 fixes, the pipeline appears to run but produces garbage classifications. Cost reduction is entirely driven by P1 fixes:

- Lag Window Rule — classifies EXPIRED_WITHDRAWN without BatchData when daysSinceRemoval > DEED_LAG_CUTOFF_DAYS(120 days). Estimated ~40% reduction in

- BatchData input.
- Delta Validation — skips re-processing listings already seen (listingId present in listing_seen_log). Reduces processing pool from ~215 records/week to ~13 new records/week after first cycle — a ~94% reduction.
 - Heuristic Scoring — routes only AMBIGUOUS records (score -1 to 0, ~10% of pool) to BatchData. All other score buckets are classified locally at zero cost.
 - Combined P1 effect: BatchData calls reduced from ~215/area to ~21/area — a 90% reduction in API spend.

Pipeline Outcome Distribution After Full Remediation

Table 11 — Estimated Record Distribution After Full Remediation

Pipeline Stage	Resolution Method	Est. % of Inactive Records	BatchData Required?
Hard signal — /properties deed lookup	Step 1 cross-reference confirms lastSaleDate within 45 days of removedDate	~35%	No
Heuristic scoring — high confidence	Score geq 3 (LIKELY_SOLD) or leq -4 (LIKELY_EXPIRED_WIT	~55%	No
Lag Window auto-reject	removedDate > DEED_LAG_CUTOFF_DAYS, no Phase 1	Subset of above	No
Ambiguous — routed to BatchData	match Heuristic score -1 to 0	~10%	Yes — BatchData async bulk

Section 7 — Architecture Overview

Pipeline Data Flow Summary

Table 12 — Pipeline Step-by-Step Data Flow

Step	Operation	Input	Output	BatchData?
Step 1	/properties ingestion (saleDateRange=1	ZIP/chunk coordinates	Confirmed Sales DB populated	No
Step 2a	80) Hard Signal Check — Phase 1 match	Inactive listing vs. Confirmed Sales DB	SOLD if deed date within 45 days of removedDate	No
Step 2b	Lag Window Rule	daysSinceRemoval vs. DEED_LAG_CUT OFF_DAYS	EXPIRED_WITHDRAWN if > 120 days, no Phase 1 match	No
Step 2c	Delta Validation	listingId vs. listing_seen_log	SKIP if previously seen; log new listings	No
Step 2d	Heuristic Scoring (scoreInactiveListing.ts)	daysOnMarket, removedDate, listedDate, lastSeenDate, history	Score bucket: LIKELY_SOLD / PROBABLY_SOLD / AMBIGUOUS / PROBABLY_EXPIRED / LIKELY_EXPIRED_WITHDRAWN	Only for AMBIGUOUS (score -1 to 0)
Step 3	BatchData Async Bulk Lookup	AMBIGUOUS records only (~10% of pool)	SOLD_CONFIRMED / PENDING / EXPIRED_WITHDRAWN_CONFIRMED via webhook	Yes

Heuristic Scoring Model Reference

Table 13 — Heuristic Scoring Signal Weights

Signal	Field(s) Used	Condition	Score	Classification Direction
Fast removal	daysOnMarket	< 30 days	+3	Sold indicator
Short listing duration	removedDate - listedDate	< 45 days	+2	Sold indicator
Gradual removal	lastSeenDate vs. removedDate	Gap geq 7 days before removedDate	+1	Sold pattern

First-time listing	history object	Single entry only	+1	Sold indicator
Extended market time	daysOnMarket	> 150 days	-3	Expired indicator
DOM boundary clustering	daysOnMarket	approx 90, 180, or 365 days (plusminus3)	-99	Expired — contract expiration boundary
Duration boundary clustering	removedDate - listedDate	approx 90, 180, or 365 days (plusminus3)	-99	Expired — listing duration matches contract terms
Abrupt removal	lastSeenDate vs. removedDate	rightarrow Same day as removedDate (gap = 0) AUTO-REJECT	-2	Withdrawn indicator
Ultra-short listing	removedDate - listedDate	< 7 days	-1	Withdrawn indicator
Repeat listing	history object	3 or more prior entries	-1	Failed attempts indicator

Table 14 — Heuristic Classification Decision Thresholds

Score	Classification	BatchData?	Confidence
geq 3	LIKELY_SOLD	No	75–85%
1–2	PROBABLY_SOLD	No	60–70%
-1 to 0	AMBIGUOUS	Yes rightarrow Step 3	N/A
-2 to -3	PROBABLY_EXPIRED	No	60–70%
leq -4	LIKELY_EXPIRED_WIT HDRAWN	No	75–85%

Database Schema Summary

Table 15 — New Database Tables Required

Table Name	Purpose	Primary Key	Key Fields
listing_seen_log	Delta validation — tracks all previously seen listing IDs to prevent reprocessing	listingId	firstSeenDate, lastProcessedDate, classification, confidence

heuristic_score_log	Audit trail for all heuristic scoring decisions; used for shadow mode comparison and	id (serial)	listingId, score, signals (JSONB), classification, batchdataResult
pending_confirmation_queue	Holds PROBABLY_SOLD and AMBIGUOUS records pending Redfin cross-check or BatchData re-evaluation	listingId	formattedAddress, classifiedAt, reCheckDate, status

Chunking Architecture Notes

The 300 sq mile geographic coverage area is processed using a grid of overlapping sub-circles. The following requirements apply to the processFetchChunk.ts implementation:

- Sub-circle geometry: For a 300 sq mile rectangular area (~17.3 times 17.3 miles), use a 4times4 grid of 16 sub-circles with radius 5 miles (~78.5 sq mi each) and ~20% overlap for full coverage with no gaps.
- Cross-chunk deduplication is mandatory: the same property will appear in multiple overlapping sub-circle results. Deduplicate by formattedAddress or property id before merging results from all sub-circles. Failure to deduplicate inflates record counts and generates duplicate BatchData calls.
- Aggregate metrics correctly: total_fetched, total_inserted, and total_api_calls must aggregate across all sub-circles, not just the last one. Do not reset counters per sub-circle.
- Per-chunk error isolation: each sub-circle fetch must be wrapped in an independent try/catch block. A failed sub-circle logs the error and continues — do not abort the entire job on a single sub-circle failure. Track failed_chunks as a separate metric.
- Coverage verification: after all chunks complete, verify that the bounding box of returned properties spans the expected geographic area to catch silent API errors that returned zero results.

Redfin Pending Discrepancy — Root Cause Summary

The Redfin 'Pending' discrepancy is a Step 2 classification error, not a data source conflict. The structural timing gap between MLS syndication (near real-time) and county deed recording (30–90 days post-closing) means a property can simultaneously appear Pending on Redfin and Inactive in RentCast.

Table 16 — Data Source Timing Comparison: Redfin vs. FirstKnock Pipeline

Data Source	Update Trigger	Latency	Confirms Sale?
Redfin (MLS syndication)	Listing agent marks Pending in MLS	Near real-time (hours)	NO — contract signed, not yet closed
FirstKnock /properties (deed records)	County clerk records the deed	30–90 days after closing	YES — legally confirmed transfer
RentCast /listings/sale Inactive heuristics	Property removed from active MLS	Real-time	NO — includes Pending, Expired, Withdrawn, Cancelled

Fix: Do not classify a property as sold until either (a) Phase 1 /properties returns a lastSaleDate for the property, or (b) BatchData confirms the sale via its own deed/public records lookup. Properties classified by heuristics only should be flagged as PROBABLY_SOLD— PENDING CONFIRMATION and held in the pending_confirmation_queue until confirmed. If RentCast shows Inactive AND Redfin shows Pending, hold in queue and re-check in 30 days.

Document generated: April 2026. Based on county deed recording research for Doña Ana County NM, Bernalillo County NM, Harris County TX, Travis County TX, and Bexar County TX. RentCast propagation lag estimates based on industry aggregator benchmarks (CoreLogic 24-hr refresh, ATTOM 48-hr refresh) plus observed end-to-end pipeline lag of 30–90 days. Supersedes all prior pipeline architecture documents.